

DIY Challenge Robot

Development & Operations Guide

ROS 2 Humble · Jetson Nano · Hesai QT64 · FAST-LIO2 · Nav2

Revision: 2026-05-18

Table of Contents

1. Repository Structure & Architecture	3
2. Package Reference	5
3. Configuration & Calibration	9
4. Device Profiles	12
5. First-Time Setup on Robot Hardware	14
6. Calibration Procedures	17
7. Competition Day Operations	21
8. Developer Workflows (Laptop)	24
9. Scripts Reference	26
10. Troubleshooting	29
11. Extending the Codebase	32

1. Repository Structure & Architecture

1.1 Workspace Layout

The repository lives at `~/ros2_ws/src/DIY-Challenge-Repo/`. The directory is itself inside the colcon workspace so that colcon build run from `~/ros2_ws/` automatically discovers all packages inside `src/`.

Directory tree

```
DIY-Challenge-Repo/
├── docs/                ← SAD, installation guide, this PDF
├── profiles/            ← Per-device environment files
├──   ├── jetson.env      ← Full hardware stack (competition)
├──   ├── raspi.env       ← Lightweight relay/health role
├──   └── laptop.env      ← Debug / replay / calibration
├── scripts/            ← Shell scripts for every workflow
├──   ├── env.sh          ← Environment bootstrap (source, not execute)
├──   ├── run_robot.sh    ← Competition launch
├──   ├── debug_robot.sh  ← Debug launch (+ RViz, verbose logs)
├──   ├── record_bag.sh   ← Record all SAD topics to rosbag
├──   ├── replay_bag.sh   ← Replay bag through software stack
├──   ├── calibrate_imu.sh ← Allan variance IMU calibration
├──   ├── calibrate_extrinsics.sh ← Lidar↔IMU extrinsic calibration
├──   ├── health_check.sh ← Pre-flight hardware + TF validation
├──   └── deploy_bundle.sh ← Push calibration/map bundle to robot
├── calibration/         ← Created by calibration scripts; .gitignored
├── src/                 ← ROS 2 packages (colcon root)
├──   ├── challenge_bringup/ ← Top-level launch + Nav2/map config
├──   ├── diy_cmd_vel_mux/  ← cmd_vel arbitration node
├──   ├── diy_estop_controller/ ← STM32 E-stop mirror node
├──   ├── diy_robot_description/ ← URDF / TF tree
├──   ├── diy_localization/ ← FAST-LIO2 + EKF1 + EKF2 + navsat
├──   └── third_party_ws/    ← Pre-built SLAM & sensor packages
├──   ├── src/FAST_LIO/
├──   ├── src/LIO-SAM/
├──   ├── src/imu_utils_ros2_humble/
├──   ├── src/lidar_imu_calib/
├──   ├── src/ndt_omp_ros2/
├──   └── src/livox_ros_driver2/
```

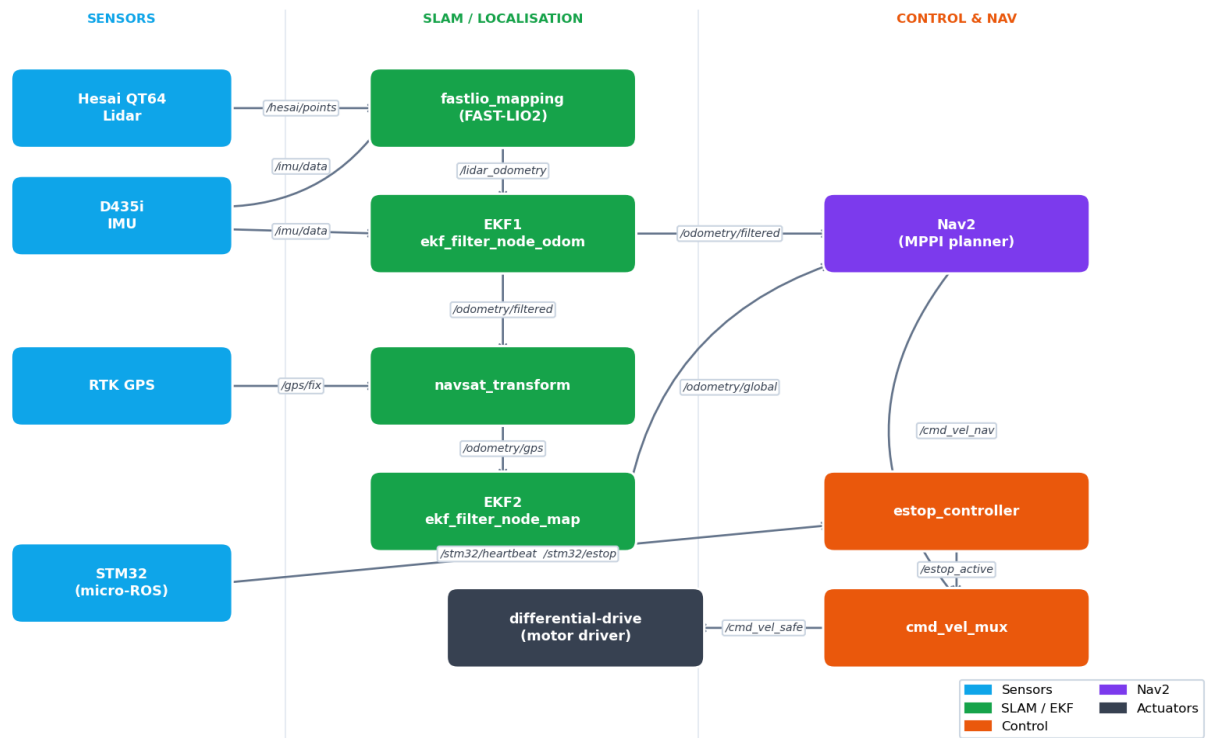
1.2 System Architecture Overview

The system is divided into three compute roles. Each role loads a matching profile which enables the appropriate set of drivers and nodes.

Role	Hardware	Responsibilities
Jetson Nano (competition)	QT64 + D435i + GPS + STM32 via micro-ROS	Lidar driver → FAST-LIO2 → EKF → Nav2 → cmd_vel_mux → motors
Raspberry Pi (optional)	STM32 via micro-ROS + GPS relay	Lightweight E-stop bridge & health monitoring only
Laptop (developer)	No hardware (bag replay or sim)	Debug, calibration, offline mapping, bag analysis

1.3 Node Communication Graph

Colour key: **blue** = sensors **green** = SLAM/EKF **orange** = control **purple** = Nav2 **dark** = actuators. Arrows are labelled with the ROS topic.



2. Package Reference

2.1 challenge_bringup

Type: ament_cmake (launch + config only) | **Depends on:** nav2_bringup, all diy_* packages

Top-level bringup package. Contains the master launch file and all Nav2 / collision-monitor configuration. This is the single entry point for every deployment — `run_robot.sh` calls `ros2 launch challenge_bringup challenge_master.launch.py`.

Key files

File	Purpose
launch/challenge_master.launch.py	Top-level launch; accepts all DIY_* flags as args
launch/joystick_drive.launch.py	Joystick teleop sub-launch (joy_node + teleop_twist_joy)
config/nav2_params.yaml	Nav2: costmaps, Smac planner, MPPI controller, behavior server
config/collision_monitor_params.yaml	Collision monitor safety zones
maps/static_map.yaml	Competition course static map (replace with your map)

2.2 diy_cmd_vel_mux

Type: ament_python | **Node:** cmd_vel_mux_node | **Executable:** cmd_vel_mux_node

Arbitrates command velocity from joystick teleop, Nav2, and an E-stop signal into a single safe `/cmd_vel_safe` output. The motor driver *only* reads `/cmd_vel_safe` — it never sees Nav2 or joystick topics directly.

Topics

Topic	Type	Direction	Description
/cmd_vel_joy	geometry_msgs/Twist	SUB	Joystick teleop output
/cmd_vel_nav	geometry_msgs/Twist	SUB	Nav2 FollowPath controller
/estop_active	std_msgs/Bool	SUB	Advisory E-stop from diy_estop_controller
/cmd_vel_safe	geometry_msgs/Twist	PUB	To motor driver (always safe/zero on E-stop)
/mux_mode	std_msgs/String	PUB	Current mode: JOYSTICK AUTONOMOUS ESTOP_LOCK

Runtime mode switch: `ros2 param set /cmd_vel_mux_node mode AUTONOMOUS (or JOYSTICK)`. `ESTOP_LOCK` cannot be set manually.

2.3 diy_estop_controller

Type: ament_python | **Node:** estop_controller_node

Mirrors the STM32 hardware E-stop state into the ROS 2 graph. **Important:** This node is advisory only — the actual motor cut happens in STM32 firmware before any ROS message is even sent. The dual-watchdog design (explicit flag + heartbeat) means the software stack goes safe even if the micro-ROS bridge drops.

WARNING: The STM32 heartbeat timeout is 800 ms. If `micro_ros_agent` crashes or the USB cable is unplugged, `/estop_active` will go True within 800 ms and `cmd_vel_mux` will publish zero velocity.

2.4 diy_robot_description

Type: ament_cmake | Provides: URDF Xacro + static TF tree

Defines the robot body geometry and all sensor placements as URDF joints. Currently all joint transforms are placeholder identity values marked **CALIB**: — they must be updated with real calibrated extrinsics before SLAM will produce accurate maps.

CALIBRATION REQUIRED: All joint xyz/rpy in urdf/robot.urdf.xacro are set to identity. Run `calibrate_extrinsics.sh` and copy the resulting transforms into the URDF before any mapping session.

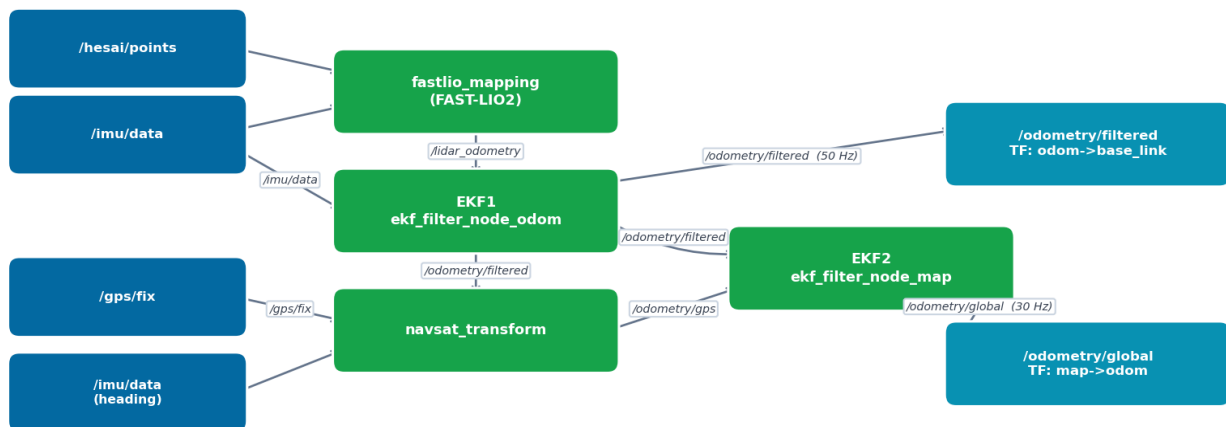
2.5 diy_localization

Type: ament_cmake (configs + launch) | **Depends on:** fast_lio, robot_localization, diy_robot_description

Launches the full localisation stack: FAST-LIO2 lidar-inertial odometry feeding two robot_localization EKF instances and a navsat_transform node for GPS fusion.

Localisation pipeline

Input topics (dark blue) feed FAST-LIO2 and the EKF nodes (green). Outputs (teal) are the topics and TF transforms consumed by Nav2.



3. Configuration & Calibration

3.1 FAST-LIO2 Config: `fast_lio_hesai_qt64.yaml`

Located at `src/diy_localization/config/fast_lio_hesai_qt64.yaml`. Configures FAST-LIO2 for the Hesai QT64 lidar.

Parameter	Value	Notes
<code>lidar_type</code>	2	PointCloud2 format (non-Livox Velodyne-style)
<code>scan_line</code>	64	QT64 has 64 beam channels
<code>scan_rate</code>	20	20 Hz output rate from QT64
<code>blind</code>	0.3	Ignore returns closer than 0.3 m (near-field noise)
<code>det_range</code>	80.0	Maximum effective range in metres
<code>extrinsic_T</code>	CALIB: [0,0,0]	Lidar→IMU translation; update after calibration
<code>extrinsic_R</code>	CALIB: identity	Lidar→IMU rotation matrix; update after calibration
<code>extrinsic_est_en</code>	true	Online estimation active until <code>lidar_imu_calib</code> completes
<code>dense_publish_en</code>	false	Bandwidth optimisation for Jetson Nano

3.2 EKF1 Local: `ekf_local.yaml`

Fuses FAST-LIO2 lidar odometry (50 Hz) with IMU angular velocity (400 Hz) in the **odom frame**. Output: `/odometry/filtered` at 50 Hz — the primary odometry source for Nav2 local MPPI controller.

- **odom0:** `/lidar_odometry` — x, y, yaw, vx, vy, vyaw used
- **imu0:** `/imu/data` — gyroscope angular velocity only (vroll, vpitch, vyaw)
- **two_d_mode:** **true** — flattens to 2D for ground robot
- **frequency:** **50 Hz** — matches FAST-LIO2 output rate

3.3 EKF2 Global: `ekf_global.yaml`

Fuses `/odometry/filtered` with GPS position in the **map frame**. Output: `/odometry/global` at 30 Hz — provides absolute position correction for long-range navigation.

NOTE: EKF2 only produces useful output when GPS has a valid fix (HDOP < 2.0). In GPS-denied environments (indoors / under trees), FAST-LIO2's internal PGO provides the map→odom transform instead.

3.4 Nav2 Configuration: `nav2_params.yaml`

Component	Plugin / Setting	Key Value
Global planner	SmacPlannerHybrid	tolerance: 0.25 m
Local controller	MPPIController	40 steps × 50 ms, batch 1000, vx_max 0.6 m/s
MPPI critics	8 critics	ObstaclesCritic, PathAlign, GoalCritic, ...
Global costmap	static+obstacle+inflation	Inflation: 0.45 m from /hesai/points
Local costmap	voxel+inflation	4 m × 4 m rolling, 0.05 m/cell
Behavior server	spin, backup, drive, wait	Recovery behaviours for Nav2 BT
BT Navigator	navigate_w_replanning_and_recovery	Default recovery BT

4. Device Profiles

The profiles system makes the same repository runnable on three different hardware configurations without modifying any source file. Each profile is a Bash environment file in **profiles/** that exports `DIY_*` variables.

How profiles work

Loading a profile manually

```
# Load the Jetson profile before launching:
source ~/ros2_ws/src/DIY-Challenge-Repo/profiles/jetson.env

# Or let env.sh auto-detect from hostname:
source ~/ros2_ws/src/DIY-Challenge-Repo/scripts/env.sh
```

Profile environment variables

Variable	Type	Purpose
DIY_ROBOT_PROFILE	string	Profile name: jetson raspi laptop
DIY_USE_HESAI	bool	Enable Hesai QT64 lidar driver node
DIY_USE_REALSENSE	bool	Enable RealSense D435i camera node
DIY_USE_GPS	bool	Enable RTK GPS & navsat_transform
DIY_USE_MICRO_ROS	bool	Enable micro-ROS agent (STM32 bridge)
DIY_USE_MOTOR_DRIVER	bool	Enable differential-drive motor node
DIY_USE_NAV2	bool	Enable Nav2 autonomous navigation stack
DIY_USE_LOCALIZATION	bool	Enable FAST-LIO2 + EKF1/EKF2
DIY_MUX_MODE	string	Startup mode for cmd_vel_mux: JOYSTICK AUTONOMOUS
DIY_FASTLIO_CONFIG	string	YAML filename in diy_localization/config/
DIY_HESAI_IP	string	Lidar IP address (default: 192.168.1.201)
DIY_MICRO_ROS_SERIAL	string	Serial device for micro-ROS (e.g. /dev/ttyACM0)
DIY_BAG_OUTPUT_DIR	path	Directory for rosbag recordings
DIY_BAG_SPLIT_DURATION	int	Bag split interval in seconds

5. First-Time Setup on Robot Hardware

5.1 Prerequisites

- Ubuntu 22.04 (Jetpack 5.x for Jetson Nano) with ROS 2 Humble installed
- micro_ros_ws built at ~/micro_ros_ws (follow micro-ROS Humble build guide)
- HesaiLidar_ROS_2.0 installed at ~/ros2_ws/src/Betsybot-Software/src/HesaiLidar_ROS_2.0/
- Git and colcon-common-extensions installed
- Python 3 with pip
- USB permissions for STM32: user added to dialout group (sudo usermod -aG dialout \$USER)
- Ethernet interface configured for lidar: static IP 192.168.1.100/24 on the lidar port

5.2 Clone and build (first time)

The repository uses **git submodules** for all six third-party packages (FAST-LIO2, LIO-SAM, imu_utils, lidar_imu_calib, livox_ros_driver2, ndt_omp_ros2). You must clone with --recurse-submodules to pull them in, or the third-party workspace build will fail with missing source errors.

Option A — automated setup (recommended)

```
# 1. Clone the repo WITH its submodules in one command
cd ~/ros2_ws/src
git clone --recurse-submodules \
    https://github.com/vasu536/DIY-Challenge-Repo.git

# 2. Run the setup script – handles patches, rosdep, and all colcon builds
cd DIY-Challenge-Repo
bash setup.sh jetson          # or: bash setup.sh laptop / raspi

# 3. After setup.sh completes, source the environment
source scripts/env.sh jetson

# 4. Verify everything is running
scripts/health_check.sh jetson
```

NOTE: setup.sh does the following automatically: initialises submodules, applies the three CMakeLists.txt build-fix patches in patches/, runs rosdep install, builds third_party_ws, then builds all five DIY packages. It is safe to run multiple times.

Option B — manual step-by-step (if setup.sh fails)

```
# 1. Clone with submodules
cd ~/ros2_ws/src
git clone --recurse-submodules \
    https://github.com/vasu536/DIY-Challenge-Repo.git
cd DIY-Challenge-Repo

# 2. Apply build-fix patches to three third-party packages
git -C third_party_ws/src/lidar_imu_calib apply ../../patches/lidar_imu_calib.patch
git -C third_party_ws/src/livox_ros_driver2 apply ../../patches/livox_ros_driver2.patch
git -C third_party_ws/src/ndt_omp_ros2 apply ../../patches/ndt_omp_ros2.patch

# 3. Source ROS 2 and install dependencies
source /opt/ros/humble/setup.bash
rosdep install --from-paths third_party_ws/src src --ignore-src -r -y

# 4. Build third-party workspace
cd third_party_ws && colcon build --symlink-install && cd ..

# 5. Source the full environment (Jetson profile)
```

```
source scripts/env.sh jetson

# 6. Build all DIY packages
cd ~/ros2_ws
colcon build --symlink-install \
  --packages-select \
    challenge_bringup \
    diy_cmd_vel_mux \
    diy_estop_controller \
    diy_robot_description \
    diy_localization

# 7. Run preflight health check
~/ros2_ws/src/DIY-Challenge-Repo/scripts/health_check.sh jetson
```

WARNING: If you cloned **WITHOUT** `--recurse-submodules`, the `third_party_ws/src/` directories will be empty. Fix it by running: `git submodule update --init --recursive`

5.3 Source overlay order (CRITICAL)

ROS 2 workspaces must be sourced in the correct order — later sources override earlier ones. If you source out of order, nodes from the wrong workspace may be invoked.

Correct overlay sequence (`env.sh` handles this automatically)

```
1. source /opt/ros/humble/setup.bash           # ROS 2 base
2. source ~/micro_ros_ws/install/setup.bash    # micro-ROS
3. source ~/ros2_ws/src/Betsybot-Software/src/HesaiLidar_ROS_2.0/install/setup.bash
4. source ~/ros2_ws/src/DIY-Challenge-Repo/third_party_ws/install/setup.bash
5. source ~/ros2_ws/install/setup.bash         # DIY packages
```

NOTE: `scripts/env.sh` performs all 5 steps automatically when sourced. Never manually source individual `setup.bash` files for production use.

5.4 Network configuration

Hesai QT64 Ethernet setup (example for `eth0`)

```
# Set a static IP on the Jetson's lidar ethernet port
sudo ip addr add 192.168.1.100/24 dev eth0
sudo ip link set eth0 up

# Verify the lidar is reachable (default QT64 IP: 192.168.1.201)
ping 192.168.1.201

# To make permanent, add to /etc/netplan/01-netcfg.yaml:
#   network:
#     ethernets:
#       eth0:
#         addresses: [192.168.1.100/24]
```

5.5 micro-ROS agent: build and first-time setup

The micro-ROS agent bridges the STM32 firmware (which runs a micro-ROS executor) to the ROS 2 network over a USB serial link. It must be built once as a separate workspace and is started automatically by **challenge_master.launch.py** (when `use_micro_ros:=true`). This section covers the one-time build and optional persistence setup.

Build `micro_ros_ws` (one-time)

Build the micro-ROS agent workspace

```
# 1. Create the workspace
mkdir -p ~/micro_ros_ws/src
```

```
cd ~/micro_ros_ws

# 2. Clone the micro-ROS agent for ROS 2 Humble
git clone -b humble https://github.com/micro-ROS/micro_ros_setup.git src/micro_ros_setup

# 3. Source ROS 2 and install dependencies
source /opt/ros/humble/setup.bash
rosdep update
rosdep install --from-paths src --ignore-src -y

# 4. Build the setup tools
colcon build --symlink-install
source install/setup.bash

# 5. Create and build the micro-ROS agent
ros2 run micro_ros_setup create_agent_ws.sh
ros2 run micro_ros_setup build_agent.sh
source install/setup.bash

# 6. Verify the agent binary is available
ros2 run micro_ros_agent micro_ros_agent --help | head -5
```

USB permissions (one-time)

The STM32 appears as `/dev/ttyACM0` (or similar). Add your user to the **dialout** group so the agent can open it without `sudo`. This only needs to be done once; log out and back in afterwards for the group change to take effect.

Grant serial port access

```
sudo usermod -aG dialout $USER
```

```
# Log out and back in, then verify:
groups | grep dialout
```

```
# Identify which device node the STM32 uses:
ls /dev/ttyACM* /dev/ttyUSB*
# (re-plug the USB cable and watch dmesg if the device does not appear)
dmesg | tail -5
```

Test: verify STM32 topics appear

Manual test before using the launch file

```
# Terminal 1 – start the agent manually
source ~/micro_ros_ws/install/setup.bash
ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyACM0 -b 921600

# Terminal 2 – check that the STM32 topics appear (within ~2 seconds)
source ~/ros2_ws/src/DIY-Challenge-Repo/scripts/env.sh jetson
ros2 topic list | grep stm32
# Expected output:
#   /stm32/heartbeat
#   /stm32/estop

# Verify heartbeat is publishing at ~10 Hz
ros2 topic hz /stm32/heartbeat
```

Optional: run agent as a systemd service

For competition use, running the agent as a systemd service means it starts automatically at boot and restarts if it crashes — no manual terminal needed.

Create `/etc/systemd/system/micro-ros-agent.service`

```
[Unit]
Description=micro-ROS Agent (STM32 bridge)
After=network.target

[Service]
User=YOUR_USERNAME
ExecStart=/bin/bash -c \
    "source /opt/ros/humble/setup.bash && \
    source /home/YOUR_USERNAME/micro_ros_ws/install/setup.bash && \
    ros2 run micro_ros_agent micro_ros_agent serial \
    --dev /dev/ttyACM0 -b 921600"
Restart=on-failure
RestartSec=3
```

```
[Install]
WantedBy=multi-user.target
```

Enable and start the service

Replace YOUR_USERNAME in the service file first, then:

```
sudo systemctl daemon-reload
sudo systemctl enable micro-ros-agent.service
sudo systemctl start micro-ros-agent.service
```

```
# Check status
sudo systemctl status micro-ros-agent.service
```

NOTE: When the agent runs as a systemd service, the challenge_master.launch.py node block for micro_ros_agent is redundant but harmless — the launch file will simply fail to bind the serial port (already in use) and the system will continue normally via the service. To avoid the conflict, set use_micro_ros:=false in the launch args when the service is active.

5.6 Accessing the Jetson Nano (SSH / headless setup)

For competition use the Jetson Nano typically runs headless (no monitor). All interaction is over SSH from a laptop on the same local network.

First-time physical setup

- **JetPack 5.x default credentials:** username ubuntu, password ubuntu (change immediately after first login)
- First boot requires a monitor + keyboard to complete the initial Ubuntu wizard — set hostname to jetson and enable OpenSSH server
- Alternative: serial console via the 40-pin header UART (115200 baud, pins 8/10/6) if no monitor is available

Finding the Jetson IP and connecting

SSH into the Jetson Nano

```
# The Jetson advertises itself via mDNS as jetson.local (same LAN / WiFi)
ping jetson.local
```

```
# If mDNS does not work, scan the subnet for the Jetson
nmap -sn 192.168.1.0/24 | grep -A2 "NVIDIA\|jetson"
```

```
# Connect (replace jetson.local with the IP if nmap was needed)
ssh ubuntu@jetson.local
```

```
# First time: change the default password immediately
passwd
```

```
# Verify ROS 2 environment is reachable
```

```
source /opt/ros/humble/setup.bash && ros2 node list
```

Passwordless SSH (recommended for scripts)

Set up key-based authentication from your laptop

```
# On your LAPTOP – generate a key if you do not have one already
ssh-keygen -t ed25519 -C "diy-challenge"
```

```
# Copy the public key to the Jetson
ssh-copy-id ubuntu@jetson.local
```

```
# Verify: this should open a shell with no password prompt
ssh ubuntu@jetson.local
```

Optional: VNC desktop session

Enable VNC for a full GUI via RealVNC or TigerVNC

```
# On the Jetson: enable the built-in VNC server (JetPack 5.x includes it)
sudo systemctl enable vncserver-x11-serviced.service
sudo systemctl start vncserver-x11-serviced.service
```

```
# On your laptop: connect with any VNC client to
# jetson.local:5900 (password set via the VNC app on first run)
# Or tunnel via SSH for security:
ssh -L 5900:localhost:5900 ubuntu@jetson.local
# then connect your VNC client to localhost:5900
```

5.7 Jetson Nano performance: power mode, clocks, and GPU

Does the navigation stack use the GPU?

Short answer: **no** — the current navigation pipeline runs entirely on the CPU. FAST-LIO2 uses an ikd-Tree with Eigen-based Kalman filter iterations; LIO-SAM uses GTSAM factor graphs with OpenMP-parallelised CPU threads; Nav2 and the EKF nodes are single-threaded ROS 2 processes. The Jetson's 128-core Maxwell GPU (CUDA 11.x) is idle during normal robot operation. It would only be used if you added a neural-network inference node (e.g. YOLO for obstacle detection via TensorRT).

Despite the GPU not being used by the stack, **performance tuning is still critical**: the quad-core Cortex-A57 runs FAST-LIO2 at ~50 Hz and all Nav2 planners simultaneously. Without locking the clocks to maximum the CPU will throttle and introduce planning jitter.

Set maximum performance mode (nvpmodel + jetson_clocks)

Run before every competition attempt

```
# Show current power mode (default at boot is often 5W / mode 1)
sudo nvpmodel -q
```

```
# Switch to MAXN (10W) mode – uses all 4 CPU cores at full speed
sudo nvpmodel -m 0
```

```
# Lock CPU, GPU, and EMC clocks to their maximum
sudo jetson_clocks
```

```
# Verify: CPU frequencies should all be at max (e.g. 1.479 GHz for Nano)
cat /sys/devices/system/cpu/cpu*/cpufreq/scaling_cur_freq
```

```
# Check overall thermal + power state with jtop (install once: pip3 install jetson-stats)
jtop
```

WARNING: nvpmodel and jetson_clocks settings reset on reboot. Add them to a systemd service or to /etc/rc.local so they are applied automatically at startup — especially important on competition day.

Make max-performance persistent across reboots

Create `/etc/systemd/system/jetson-maxperf.service`

```
[Unit]
Description=Set Jetson to MAXN power mode and lock clocks
After=multi-user.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/bin/sh -c "nvpmodel -m 0 && jetson_clocks"

[Install]
WantedBy=multi-user.target
```

Enable the service

```
sudo systemctl daemon-reload
sudo systemctl enable jetson-maxperf.service
sudo systemctl start jetson-maxperf.service
```

```
# Check it ran successfully
sudo systemctl status jetson-maxperf.service
```

Add a swap file (recommended; Jetson Nano has only 4 GB RAM)

Create a 4 GB swap file — run once

```
# Create the swap file
sudo fallocate -l 4G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile

# Make permanent across reboots
echo '/swapfile swap swap defaults 0 0' | sudo tee -a /etc/fstab

# Verify
free -h | grep Swap
```

NOTE: The swap file is especially helpful during colcon builds of FAST-LIO2 and LIO-SAM. At runtime the system should stay within physical RAM; if tegrastats shows heavy swap usage during operation the robot has too many nodes running simultaneously.

Monitor CPU/GPU during a run

Real-time monitoring tools

```
# tegrastats (built-in, prints every 1 second)
tegrastats --interval 1000

# jtop (rich TUI dashboard – install once)
pip3 install jetson-stats
sudo systemctl restart jtop.service # needed after install
jtop

# Key things to watch in tegrastats output:
# RAM   xxx/3964MB – keep below ~3 GB during operation
# CPU   [xx% xx% xx% xx%] – should all be near 100% during SLAM
# GPU   xx% – will be near 0% (navigation stack is CPU-only)
# Temp  SOC xx.xC – keep below 80°C; throttling starts at 85°C
```

Verify CUDA installation (optional)

Confirm CUDA 11.x is present from JetPack

Check CUDA compiler version

```
nvcc --version
# Expected: Cuda compilation tools, release 11.4 (or similar JetPack 5.x version)

# Check GPU is visible to CUDA
python3 -c "import subprocess; subprocess.run(['nvidia-smi'])"
# Or:
ls /dev/nvidia*    # should list /dev/nvidia0

# If you later add TensorRT / YOLO inference:
python3 -c "import tensorrt; print(tensorrt.__version__)"
python3 -c "import torch; print(torch.cuda.is_available())"
```

6. Calibration Procedures

Calibration is required before the first mapping session and whenever a sensor is physically moved or replaced. The scripts in `scripts/` automate each step. Always calibrate in this order:

- Step 1: IMU Allan Variance (`calibrate_imu.sh`) — 2–3 hours
- Step 2: Lidar↔IMU extrinsics (`calibrate_extrinsics.sh`) — 30 minutes
- Step 3: Update YAML + URDF with results
- Step 4: Prior map generation session (`offline_mapping.launch.py`)

6.1 IMU Allan Variance Calibration

Measures IMU noise density (white noise σ) and bias instability (random walk) from a 2–3 hour static recording. Outputs are used in FAST-LIO2 config as **`gyr_cov`**, **`acc_cov`**, **`b_gyr_cov`**, **`b_acc_cov`**.

Run IMU calibration

```
# Ensure robot is on a stable, vibration-free surface (e.g. foam pad)
# Hesai lidar and all motors must be OFF during recording
```

```
source ~/ros2_ws/src/DIY-Challenge-Repo/scripts/env.sh jetson
```

```
# Start the IMU source (micro-ROS agent must be running)
# Then in a second terminal:
./scripts/calibrate_imu.sh jetson --duration 7200
```

```
# After completion, find the result YAML in calibration/
# Copy gyr_n, gyr_w, acc_n, acc_w into:
#   src/diy_localization/config/fast_lio_hesai_qt64.yaml
```

CALIBRATION REQUIRED: After IMU calibration, set `extrinsic_est_en: false` only AFTER completing Step 2 (extrinsics). Leave it true until both calibrations are done so FAST-LIO2 can self-correct.

6.2 Lidar↔IMU Extrinsic Calibration

Calibrates the rigid transform (translation + rotation) between the lidar sensor frame and the IMU sensor frame. Requires 2 minutes of figure-8 excitation motion.

Run extrinsic calibration

```
# Robot must be mobile. Drive a figure-8 (≥ 2 m diameter circles) for 2 min.
# The script records a bag then runs lidar_imu_calib solver automatically.
```

```
./scripts/calibrate_extrinsics.sh jetson --duration 120
```

```
# After completion:
# 1. Open calibration/lidar_imu_extrinsics_<timestamp>.txt
# 2. Copy extrinsic_T and extrinsic_R into fast_lio_hesai_qt64.yaml
# 3. Update joint xyz/rpy in urdf/robot.urdf.xacro
# 4. Set extrinsic_est_en: false in fast_lio_hesai_qt64.yaml
# 5. Rebuild: colcon build --packages-select diy_localization diy_robot_description
```

6.3 Prior Map Generation

After sensor calibration, drive the competition course once to build a prior map using LIO-SAM. This map is used by Nav2 as the global static layer.

Offline mapping session

```
# Start the offline_mapping.launch.py (LIO-SAM with RViz)
source scripts/env.sh jetson
```



```
ros2 launch diy_localization offline_mapping.launch.py use_rviz:=true

# Drive the entire competition course manually
# Watch RViz to confirm loop-closure events (green segments)

# When done, save the map:
ros2 service call /lio_sam/save_map lio_sam/srv/SaveMap \
    "{resolution: 0.1, destination: '/tmp/competition_map'}"

# Copy map files to challenge_bringup/maps/:
cp /tmp/competition_map/GlobalMap.pcd src/challenge_bringup/maps/
# Convert to Nav2 occupancy grid:
ros2 run nav2_map_server map_saver_cli -f src/challenge_bringup/maps/static_map
```

7. Competition Day Operations

7.1 Pre-flight checklist

Run `health_check.sh` before every run. It validates:

- TF tree completeness (map → odom → base_link → lidar_link)
- Critical topic rates (/hesai/points ≥ 10 Hz, /imu/data ≥ 150 Hz)
- E-stop state (/estop_active = false)
- Nav2 lifecycle manager nodes active
- Battery voltage within operating range (via /diagnostics)

Pre-flight

On Jetson, run before every competition attempt:

```
./scripts/health_check.sh jetson
```

Expected output: all checks PASS

If any check FAILS, do NOT start the run until resolved

7.2 Competition launch sequence

Standard competition run

Terminal 1: Source environment

```
source ~/ros2_ws/src/DIY-Challenge-Repo/scripts/env.sh jetson
```

Terminal 2: Start the full competition stack

```
./scripts/run_robot.sh jetson
```

Wait for FAST-LIO2 to initialise (typically 5-10 s after lidar starts)

Monitor /mux_mode — should show AUTONOMOUS when ready

To switch to manual joystick at any time:

```
ros2 param set /cmd_vel_mux_node mode JOYSTICK
```

To return to autonomous:

```
ros2 param set /cmd_vel_mux_node mode AUTONOMOUS
```

7.3 Recording during a run

Record competition bag (separate terminal)

Start recording BEFORE launching the stack (captures all startup messages)

```
./scripts/record_bag.sh jetson --label competition_run1
```

Or add a custom label to describe the specific scenario:

```
./scripts/record_bag.sh jetson --label east_gate_approach
```

NOTE: Bags are written to `DIY_BAG_OUTPUT_DIR` (/mnt/usb/bags on Jetson). Ensure the USB SSD is mounted before recording. Bag files are split every 5 minutes (300 s) and compressed with `zstd`.

7.4 Emergency procedures

- **Hardware E-stop button pressed:** Motors cut immediately by STM32. ROS sees /estop_active = True within one publish cycle. To resume: physically release the E-stop button, then verify /estop_active goes False, then issue `ros2 param set /cmd_vel_mux_node mode AUTONOMOUS`.
- **Nav2 stuck / loop:** Switch to JOYSTICK mode, manually drive clear, then switch back to AUTONOMOUS.
- **FAST-LIO2 diverged (map drifting):** Restart the localisation launch with `ros2 launch diy_localization localization.launch.py`. FAST-LIO2 will reinitialise from the next lidar scan.

- **micro-ROS connection lost:** Reconnect USB, then `ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyACM0`.

8. Developer Workflows (Laptop)

8.1 Replaying a robot bag on laptop

The replay workflow lets you run the full software stack against recorded sensor data on the laptop without any hardware. This is the primary debugging and algorithm-tuning workflow.

Replay workflow

```
# Copy a bag from the robot:
scp -r ubuntu@jetson.local:/mnt/usb/bags/jetson_run1_<timestamp> ~/bags/

# Replay at full speed with Nav2 and RViz:
./scripts/replay_bag.sh ~/bags/jetson_run1_<timestamp> laptop

# Replay at half speed for detailed analysis:
./scripts/replay_bag.sh ~/bags/jetson_run1_<timestamp> laptop --rate 0.5

# Replay without Nav2 (SLAM-only debugging):
./scripts/replay_bag.sh ~/bags/jetson_run1_<timestamp> laptop --no-nav2
```

8.2 Isolating a single node for debugging

Debug a single node

```
# Example: debug cmd_vel_mux in isolation
./scripts/debug_robot.sh laptop --node diy_cmd_vel_mux/cmd_vel_mux_node

# While the node runs, inject test commands from another terminal:
ros2 topic pub /cmd_vel_nav geometry_msgs/Twist '{linear: {x: 0.3}}'
ros2 topic pub /estop_active std_msgs/Bool 'data: true'
ros2 topic echo /cmd_vel_safe
ros2 topic echo /mux_mode
```

8.3 Deploying updates to the robot

Deploy calibration bundle

```
# After updating calibration YAMLs or maps on the laptop:
./scripts/deploy_bundle.sh jetson.local --user ubuntu

# Dry-run first to preview what will be transferred:
./scripts/deploy_bundle.sh jetson.local --dry-run

# After deploy, rebuild on the robot:
ssh ubuntu@jetson.local 'cd ~/ros2_ws && colcon build --symlink-install \
  --packages-select diy_localization diy_robot_description challenge_bringup'
```

9. Scripts Reference

Script	Usage	Purpose
env.sh	source scripts/env.sh [profile]	Bootstrap: sources all ROS 2 overlays and loads DIY_* profile variables. Must be sourced (not executed) before any other script.
run_robot.sh	./scripts/run_robot.sh [profile]	Launches the full competition stack. Calls challenge_master.launch.py with all DIY_* flag overrides from the active profile.
debug_robot.sh	./scripts/debug_robot.sh [profile] [--node pkg/exe]	Same as run_robot but enables RViz and sets log level to DEBUG. The --node flag runs a single node in isolation for quick testing.
record_bag.sh	./scripts/record_bag.sh [profile] [--label tag]	Records all SAD-required topics to a timestamped rosbag. Automatically splits bags every DIY_BAG_SPLIT_DURATION seconds.
replay_bag.sh	./scripts/replay_bag.sh <bag> [profile] [--rate N]	Replays a bag through the software stack at optional speed multiplier. Hardware driver nodes are skipped automatically.
calibrate_imu.sh	./scripts/calibrate_imu.sh [profile] [--duration N]	2-3 hour static IMU recording followed by imu_utils Allan variance analysis. Copy the output noise params into fast_lio_hesai_qt64.yaml (gyr_cov, acc_cov, b_gyr_cov, b_acc_cov).
calibrate_extrinsic s.sh	./scripts/calibrate_extrinsic.sh [profile]	Figure-8 motion recording followed by lidar_imu_calib solver. Copy extrinsic_T and extrinsic_R results into the FAST-LIO2 config and the robot URDF joint offsets.
health_check.sh	./scripts/health_check.sh [profile]	Pre-flight validation: checks ROS environment, required node liveness, topic rates, TF tree completeness, E-stop state, and Nav2 lifecycle state. Exits non-zero if any check fails.
deploy_bundle.sh	./scripts/deploy_bundle.sh <host> [--user u]	rsyncs the calibration/, maps/, and config/ directories to the robot over SSH. Run after any calibration update or map rebuild.

10. Troubleshooting

FAST-LIO2 not publishing /lidar_odometry

- Verify /hesai/points is being published: `ros2 topic hz /hesai/points`
- Check `lidar_type` in `fast_lio_hesai_qt64.yaml` — must be 2 for PointCloud2
- Confirm QT64 IP is reachable: `ping 192.168.1.201`
- Check FAST-LIO2 logs: `ros2 node info /fastlio_mapping`

/estop_active stays True after releasing the button

- Check if micro-ROS heartbeat is arriving: `ros2 topic hz /stm32/heartbeat`
- If no heartbeat: verify micro-ROS agent is running and USB is connected
- Check `/estop_reason` topic to see which condition is firing
- If STM32 flag: check `/stm32/estop_active` topic value

Nav2 not producing goals / robot not moving

- Verify `/odometry/filtered` is publishing at ~50 Hz
- Check TF tree: `ros2 run tf2_tools view_frames`
- Ensure `map→odom` transform exists (from FAST-LIO2 or EKF2)
- Check Nav2 lifecycle nodes: `ros2 lifecycle get /planner_server`
- Verify `/mux_mode = AUTONOMOUS`: `ros2 topic echo /mux_mode`

colcon build fails on diy_localization / diy_robot_description

- Source environment before building: `source scripts/env.sh jetson`
- Check `third_party_ws` is built: `ls third_party_ws/install/`
- Run with verbose output: `colcon build --event-handlers console_direct+`
- Check for missing dependencies: `rosdep check --from-paths src/`

Joystick not working / cmd_vel not passing through

- Verify joystick device: `ls /dev/input/js*`
- Check `/mux_mode = JOYSTICK` (not AUTONOMOUS or ESTOP_LOCK)
- Verify `/cmd_vel_joy` is publishing: `ros2 topic hz /cmd_vel_joy`
- Check `joy_staleness_timeout_s` parameter (default 2 s)

GPS not converging / EKF2 drifting

- Check `/gps/fix` type — must be type 2 (DGPS/RTK) for accurate fusion
- Verify `magnetic_declination_radians` in `navsat_transform.yaml` for your location
- Allow 30+ seconds after startup for EKF2 heading initialisation
- Check `/gps/filtered` topic to see filtered GPS in map frame

11. Extending the Codebase

11.1 Adding a new sensor driver

- Add the driver node to `challenge_master.launch.py` with an `IfCondition` guard and a matching `DIY_USE_` variable in each `profiles/*.env` file
- Add the sensor link to `urdf/robot.urdf.xacro` with a `CALIB: joint`
- Add the sensor topic to `record_bag.sh` `REQUIRED_TOPICS` list
- Add a topic-rate check in the `HEALTH CHECKS` section of `health_check.sh`

11.2 Adding a new ROS 2 package

Create a new `ament_python` package

```
cd ~/ros2_ws/src/DIY-Challenge-Repo/src
ros2 pkg create my_new_package \
  --build-type ament_python \
  --dependencies rclpy std_msgs geometry_msgs
# Add your node, update setup.py entry_points, colcon build
```

11.3 Swapping the SLAM algorithm

To swap FAST-LIO2 for another algorithm (e.g. LOAM, LIO-SAM for runtime), update `localization.launch.py` to launch the new node and remap its odometry output to `/lidar_odometry`. EKF1 will pick it up automatically since it subscribes by topic name.

11.4 Updating Nav2 parameters

All Nav2 config lives in `src/challenge_bringup/config/nav2_params.yaml`. Key values to tune for a new course:

- `vx_max / wz_max` — speed limits (respect course width and corner radii)
- `inflation_radius` — conservative by default at 0.45 m; reduce if course has narrow passages (minimum = `robot_radius + 0.05 m`)
- `batch_size` — increase MPPI batch to 2000 for smoother paths if Jetson CPU allows
- `xy_goal_tolerance` — 0.20 m default; increase to 0.35 m for loose waypoint following

End of Document

DIY-Challenge-Repo | ROS 2 Humble | Generated 2026-05-18 | For questions, see [docs/diy-sad.html](#) or the repo README.